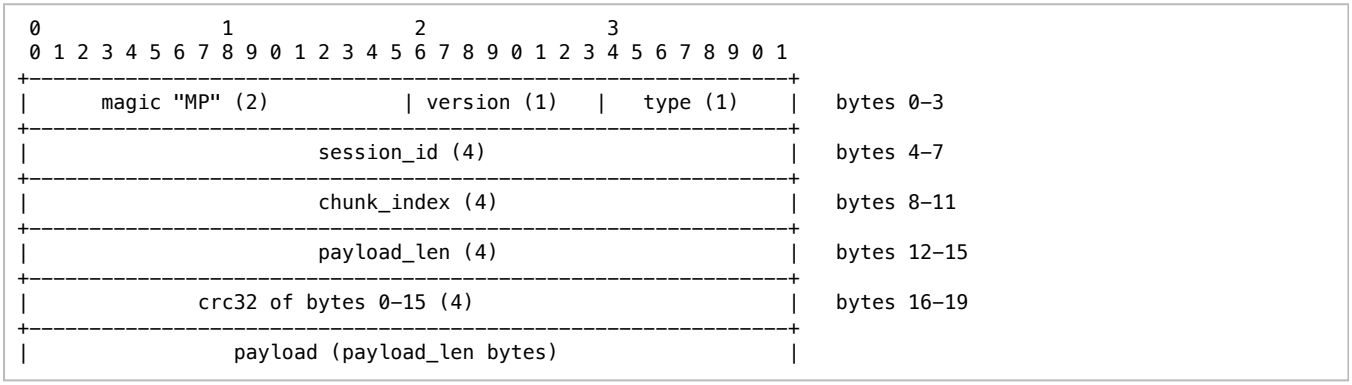
MPFT Protocol

MPFT runs over ordinary TCP connections. Each TCP connection is a **path**. All integers are big-endian. Implementation: `mpft/protocol.py` (framing), `mpft/sender.py`, `mpft/receiver.py`.

1. Frame format

Every message is a 20-byte header followed by `payload_len` bytes of payload.



Field	Size	Meaning
magic	2	ASCII `MP`; anything else is rejected
version	1	1
type	1	message type, table below
session_id	4	random 32-bit id chosen by the sender, identifies one transfer on all its paths
chunk_index	4	chunk number for CHUNK/ACK/NACK, 0 otherwise
payload_len	4	payload bytes after the header (limit 64 MiB)
crc32	4	CRC-32 of the first 16 header bytes (detects a damaged header)

2. Message types

Type	Name	Direction	Payload
1	HELLO	sender to receiver	none; first message on every new connection, carries the session id
2	MANIFEST	sender to receiver	JSON `{name, size, chunk_size, total_chunks, file_sha256, chunk_sha256[]}`
3	MANIFEST_ACK	receiver to sender	JSON list of chunk indexes the receiver already holds (resume)
4	CHUNK	sender to receiver	raw bytes of chunk `chunk_index`
5	ACK	receiver to sender	none; `chunk_index` = chunk stored and verified
6	NACK	receiver to sender	none; `chunk_index` = chunk failed verification, resend

Type	Name	Direction	Payload
7	DONE	both	sender to receiver: "all chunks acknowledged, please verify". Receiver to sender: "file verified and renamed"
8	ERROR	receiver to sender	text reason (no manifest, hash mismatch, incomplete)

Chunk `i` covers file bytes `[i*chunk_size, min((i+1)*chunk_size, size))`. The last chunk may be shorter.

3. Sequence of a transfer (2 paths)

```
Sender (path 0)        Sender (path 1)                      Receiver
     | --- HELLO(sid) ------------------------------------------>|
     | --- MANIFEST(json) --------------------------------------->|  open file.part, scan it (resume)
     |<-------------------------------- MANIFEST_ACK([held idx]) ---|
     |                          | --- HELLO(sid) ---------------->|
     | --- CHUNK 0 ---------------------------------------------->|  check len + SHA-256, pwrite
     |                          | --- CHUNK 1 ------------------->|
     |<------------------------------------------------ ACK 0 ---|
     |                          |<------------------------ ACK 1 ---|
     | --- CHUNK 2 (corrupted) ---------------------------------->|  SHA-256 mismatch
     |<------------------------------------------------ NACK 2 ---|
     | --- CHUNK 2 (retransmit) --------------------------------->|
     |<------------------------------------------------ ACK 2 ---|
     |    ... all chunks ACKed ...                                |  last chunk: whole-file SHA-256
     | --- DONE ------------------------------------------------->|  verify, rename file.part -> file
     |<------------------------------------------------ DONE -|
```

Whichever path connects first sends the MANIFEST; the other paths wait until the MANIFEST_ACK is known (`Sender.ensure_manifest`). Each path then keeps exactly one CHUNK in flight (stop-and-wait per path).

4. State machines

Sender, per path worker (`Sender.worker`):

```
CONNECT --ok--> HANDSHAKE --> IDLE --pop chunk--> SENDING --ACK--> IDLE
   |  fail x4      |  fail                          |  NACK: requeue chunk, stay connected
   v               v                                |  timeout / reset / bad frame:
 DOWN <---- backoff + reconnect <--------------------+  requeue chunk, close, reconnect
 (after 6 consecutive failures the path stays DOWN)
any state --all chunks ACKed or fatal error--> STOP
```

Sender, whole transfer (`Sender.run`): `HASHING -> TRANSFERRING -> all ACKed -> DONE handshake -> SUCCESS`, or `-> FAILED` if a chunk exceeds 5 retries, all paths are down, or the receiver answers ERROR.

Receiver, per connection (`ReceiverHandle._serve`):

```
WAIT_HELLO -> READY --MANIFEST--> (open/scan session, reply MANIFEST_ACK) --> READY
READY --CHUNK--> verify -> ACK | NACK --> READY
READY --DONE---> finalize -> DONE | ERROR --> READY
any bad frame (magic, version, CRC, length) or socket error -> close connection
```

Receiver, per session (`Session`): `PENDING --last missing chunk stored--> whole-file SHA-256 --> VERIFIED (file.part renamed)` or `MISMATCH` (file.part kept, never renamed).

5. Error rules

1. A frame with bad magic, version, unknown type, header CRC mismatch or `payload_len > 64 MiB` is a `ProtocolError`: the stream is out of sync, so that connection is closed. The sender treats it as a path failure.
2. A CHUNK whose index is out of range, whose length is wrong or whose SHA-256 differs from the manifest is answered with NACK; nothing is written.
3. A duplicate CHUNK (already stored) is re-ACKed and not rewritten, so a lost ACK is harmless.
4. Sender: NACK, timeout (default 5 s), reset or invalid reply requeues the chunk. Retries use exponential backoff `min(0.05 * 2^(n-1), 1.0)` s and a retried chunk goes to the front of the queue. More than 5 retries for one chunk fails the transfer.
5. A path that fails reconnects with backoff `min(0.1 * 2^fails, 2)` s; after 6 consecutive failures it is DOWN. Each connect attempt tries 4 times (0.2, 0.4, 0.8, 1.6 s waits).
6. CHUNK before MANIFEST for that session id is answered with ERROR.
7. The receiver renames `name.part` to `name` only after the whole-file SHA-256 equals the manifest value.
8. Resume: on MANIFEST the receiver re-hashes every chunk already in `name.part` and reports only those that match, so damaged or partial chunks are simply sent again.
9. The file name in the manifest is reduced to its base name before use (no path traversal).
10. Integrity layers: TCP checksum (network), header CRC-32 (framing), per-chunk SHA-256 (data, enables selective retransmit), whole-file SHA-256 (final proof).